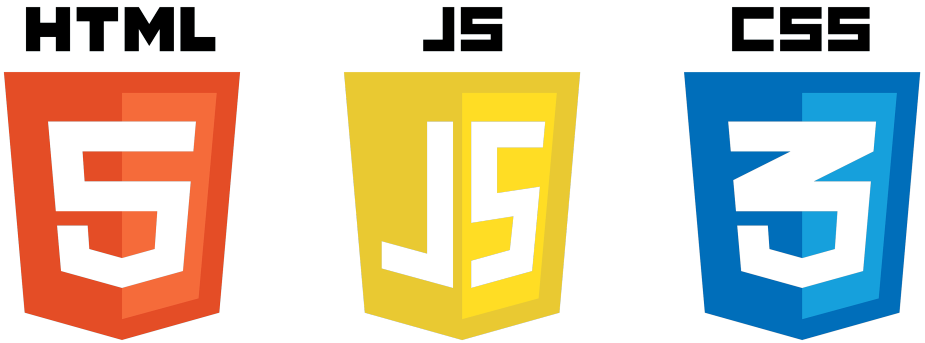
# JSON en `fetch()`



# JSON

# Waarvoor gebruiken we data?

Webapplicaties werken met data.

Voorbeelden:

- berichten op sociale media
- producten in een webshop
- gebruikersgegevens

We moeten deze data:

- opslaan
- versturen
- ophalen

# Data uitwisselen

Data wordt vaak uitgewisseld tussen:

- browsers
- servers
- externe API's

Hiervoor hebben we een standaard formaat nodig, zoals:

- plaintext
- CSV
- XML
- JSON

# Voorbeeld: plaintext

```
Milan heeft 5 likes
Sophie heeft 12 likes
```

- zeer eenvoudig formaat
- geen vaste structuur
- moeilijk automatisch te verwerken door computers

# Voorbeeld: CSV (Comma Separated Values)

```
user,likes
Milan,5
Sophie,12
```

- eenvoudig en compact formaat
- geschikt voor tabellen (rijen en kolommen)
- moeilijk te gebruiken bij complexe data

# Voorbeeld: XML (eXtensible Markup Language)

```
<users>
  <user>
    <name>Milan</name>
    <likes>5</likes>
  </user>
  <user>
    <name>Sophie</name>
    <likes>12</likes>
  </user>
</users>
```

- duidelijk gestructureerd met tags
- geschikt voor complexe data
- veel herhaling $\rightarrow$ langer en moeilijker leesbaar

# Voorbeeld: JSON (JavaScript Object Notation)

```
{
  "users": [
    {
      "name": "Milan",
      "likes": 5
    },
    {
      "name": "Sophie",
      "likes": 12
    }
  ]
}
```

- duidelijk gestructureerd met key-value paren
- compacter dan XML $\rightarrow$ minder tekst nodig
- lijkt sterk op JavaScript objecten

# XML vs JSON

**XML**

```xml
<users>
  <user>
    <name>Milan</name>
    <likes>5</likes>
  </user>
  <user>
    <name>Sophie</name>
    <likes>12</likes>
  </user>
</users>
```

Lengte (zonder whitespace): 111

**JSON**

```json
{
  "users": [
    {
      "name": "Milan",
      "likes": 5
    },
    {
      "name": "Sophie",
      "likes": 12
    }
  ]
}
```

Lengte (zonder whitespace): 67

# Wat is JSON?

JSON (JavaScript Object Notation) is een formaat om data op te slaan en uit te wisselen.

Eigenschappen:

- lijkt sterk op een JavaScript object
- bestaat uit key-value paren (keys zijn altijd strings)
- waarden kunnen zijn:
  - strings
  - numbers
  - booleans
  - arrays
  - objecten

# Waarom JSON?

JSON is vandaag het meest gebruikte formaat in webontwikkeling.

Vaak gebruikt voor:

- data uitwisselen tussen server en browser
- configuratiebestanden
- API-communicatie

Voordelen:

- eenvoudig en leesbaar
- compact (minder data om te versturen)
- gemakkelijk te gebruiken in code
- geschikt voor complexe data

# JSON vs JavaScript

**JSON**

- alleen data, geen code
- keys altijd tussen " "
- geen trailing comma

```
{
  "users": [
    {
      "name": "Milan",
      "likes": 5
    },
    {
      "name": "Sophie",
      "likes": 12
    }
  ]
}
```

**JavaScript**

- code mogelijk, bv. functies
- keys mogen zonder quotes
- trailing comma toegestaan

```
{
  users: [
    {
      name: "Milan",
      likes: 5
    },
    {
      name: "Sophie",
      likes: 12,
    }
  ]
}
```

# JSON ↔ JavaScript

- JSON → JavaScript: `JSON.parse()`

```
const json = '{"user":"Milan","likes":5}';
const obj = JSON.parse(json);
```

- JavaScript → JSON: `JSON.stringify()`

```
const obj = {
  name: "Milan",
  likes: 5
};
const json = JSON.stringify(obj);
```

## *VHSI Connect* – Opdracht 1

In de JavaScript staat er hardgecodeerde data (`posts`).

Zet dit object om naar een JSON-bestand.

- Noem het bestand `posts.json`
- Plaats het in dezelfde map als `vhsi-connect.js`

```
fetch()
```

# Wat is `fetch()`?

`fetch()` haalt data op.

Waar vandaan?

- een lokaal bestand
- een server

Welke data?

- JSON
- XML
- plaintext

# Voorbeeld: lokaal JSON-bestand laden

```
fetch("posts.json")
    .then(response => response.json())
    .then(posts => {
        // alleen hier kan je de data gebruiken
    });
```

- `fetch("posts.json")`
  - haalt het bestand `posts.json` op
  - werkt *asynchroon* (niet meteen klaar)

# Voorbeeld: lokaal JSON-bestand laden

```
fetch("posts.json")
    .then(response => response.json())
    .then(posts => {
        // alleen hier kan je de data gebruiken
    });
```

- `fetch("posts.json")`
  - haalt het bestand `posts.json` op
  - werkt *asynchroon* (niet meteen klaar)
- `.then(...)`
  - wacht tot de vorige stap gedaan is

# Voorbeeld: lokaal JSON-bestand laden

```
fetch("posts.json")
    .then(response => response.json())
    .then(posts => {
        // alleen hier kan je de data gebruiken
    });
```

- `fetch("posts.json")`
  - haalt het bestand `posts.json` op
  - werkt *asynchroon* (niet meteen klaar)
- `.then(...)`
  - wacht tot de vorige stap gedaan is
- `response.json()`
  - zet JSON om naar een JavaScript object

# Voorbeeld: lokaal JSON-bestand laden

```
fetch("posts.json")
    .then(response => response.json())
    .then(posts => {
        // alleen hier kan je de data gebruiken
    });
```

- `fetch("posts.json")`
  - haalt het bestand `posts.json` op
  - werkt *asynchroon* (niet meteen klaar)
- `.then(...)`
  - wacht tot de vorige stap gedaan is
- `response.json()`
  - zet JSON om naar een JavaScript object
- `posts`
  - is een array van objecten

## *VHSI Connect* – Opdracht 2

Gebruik `fetch()` om `posts.json` in te laden.

## *VHSI Connect* – Opdracht 3

Gebruik `fetch()` om een online JSON-bestand in te laden:

`https://maximedebosschere.com/posts.json`

Vragen?

maxime.debosschere@ugent.be